summar

1.

阶段一：SQL 算子精准定位与测试样例生成流程框架

在 SQL 智能教学系统中，**阶段一（Observe：证据采集与感知）** 的核心目标是：在学生提交 SQL 作答后，系统如何通过 AST 传感器、测试样例自动生成（ParSEval 算法）以及变分隔离测试（Mutation Testing），**精准定位到具体错误的 SQL 算子并动态生成具有针对性的测试数据**。

图例说明 (Legend)

- **I (Input)**: 输入数据结构 / 变量。
- **T (Tool / Technology / Formula / Algorithm)**: 执行代码模块、核心算法或数学公式。
- **O (Output)**: 输出数据结构 / 结果变量。

视图

```
flowchart TD
    %% Phase 1: Input & Parser
    SUBMIT(["学生提交作答"]) --> I_INPUT["I: 输入数据 (student_sql, correct_sql, schema_text)"]
    I_INPUT --> T_AST_PARSE["T: AST 语法树解析验证"]

    %% Syntax Error Branch
    T_AST_PARSE -->|解析失败| O_SYNTAX_ERR["O: 语法解析错误记录"]
    O_SYNTAX_ERR --> EXIT_ERR(["结束: 直接返回语法错误"])

    %% Successful Parsing leads to Parallel Sensors
    T_AST_PARSE -->|解析成功| AST_ROOT["AST 抽象语法树"]

    AST_ROOT --> T_AST_SENSOR["T: AST 结构传感器"]
    T_AST_SENSOR --> O_AST_FEAT["O: E_AST 结构特征对比与子句差分表"]

    AST_ROOT --> T_CONSTRAINT_EXT["T: 谓词/约束抽取"]
    T_CONSTRAINT_EXT --> O_CONSTRAINTS["O: constraints 列表"]

    %% Phase 2: Dynamic Data Generation
    %% 修复点1: 在第一次出现 T_DATA_GEN 时就定义它的文本，避免飞书解析器丢失节点
    I_INPUT -.->|"提供 Schema 结构"| T_DATA_GEN["T: 拓扑对齐与动态造数"]
    O_CONSTRAINTS --> T_DATA_GEN

    %% 修复点2: 连线上带有英文括号和斜杠时，必须用双引号包裹起来，否则飞书极易报错截断
    AST_ROOT -->|"提取算子特征 (Distinct/Group/Having/Join等)"| T_TACTIC_SELECT["动态匹配选择造数策略与探针"]
    T_TACTIC_SELECT --> T_DATA_GEN
    T_DATA_GEN --> O_MOCK_DB["O: E_db 动态测试数据集"]

    %% Phase 3: Sandbox Run
    O_MOCK_DB --> T_SANDBOX["T: 内存沙盒执行与等价性判定"]
    T_SANDBOX --> O_EXEC_RES["O: E_data 沙盒执行输出与等价判定"]

    %% Phase 4: Mutation Testing (Conditional on Equivalence)
    O_EXEC_RES -->|"等价性为 False 时启动"| T_MUTATION["T: 变分隔离测试"]

    AST_ROOT -.->|"提供语法树进行变体比对"| T_COMPARE_STRUCT["比对标准与学生 AST 结构差异"]
```

2.

二、动态造数策略完备性专项检验

验证两个层面：

1. **策略完备性**：生成的数据必须包含能区分标准 SQL 与学生 SQL 的攻击样本，例如三态边界、重复探针、JOIN 键漂移、悬浮元组或聚合边界组。
2. **实现完备性**：实际后端 `generate_and_compare` 必须在这些数据上判定不等价，并产出非空且命中预期 KP 的归因。

策略板块	沙盒等价	期望 KP	实际 KP	策略检查	结论
WHERE 数值边界三态	`False`	`where`	`where`	`PASS; PASS`	`PASS`
SELECT 投影列完整性	`False`	`select-basic`	`select-basic`	`PASS; PASS; PASS`	`PASS`
	`False`	`comp-null`			`PASS`

NULL 空值过滤探针			`comp-null, where`	`PASS; PASS`	
DISTINCT 去重探针	`False`	`distinct`	`distinct`	`PASS; PASS`	`PASS`
JOIN 拓扑对齐与跨键漂移	`False`	`join-on`	`join-on`	`PASS; PASS`	`PASS`
LEFT JOIN 悬浮元组	`False`	`join-left`	`join-left, join-inner`	`PASS; PASS`	`PASS`
GROUP BY 分组粒度错	`False`	`group-by`	`group-by`	`PASS`	`PASS`
HAVING SUM 聚合边界三态	`False`	`having`	`having, group-by`	`PASS; PASS`	`PASS`
HAVING AVG 聚合边界三态	`False`	`having`	`having, group-by`	`PASS; PASS`	`PASS`
HAVING MIN 聚合边界三态	`False`	`having`	`having, group-by`	`PASS; PASS`	`PASS`
HAVING MAX 聚合边界三态	`False`	`having`	`having, group-by`	`PASS; PASS`	`PASS`
HAVING COUNT 组大小三态	`False`	`having`	`having, group-by`	`PASS; PASS`	`PASS`
ORDER BY 有序精确比对	`False`	`order-by`	`order-by`	`PASS; PASS`	`PASS`
LIMIT 行数边界	`False`	`limit`	`limit`	`PASS; PASS`	`PASS`
子查询内外层值域重合	`False`	`where`	`where`	`PASS; PASS`	`PASS`
相关子查询内外层关联	`False`	`subquery-correlated`	`where, subquery-correlated`	`PASS; PASS; PASS`	`PASS`
集合操作 UNION 去重差异	`False`	`union`	`union`	`PASS`	`PASS`
集合操作 INTERSECT 交集差异	`False`	`intersect`	`intersect`	`PASS`	`PASS`
集合操作 EXCEPT 排他差异	`False`	`except`	`except, where`	`PASS`	`PASS`
CASE WHEN 分支边界三态	`False`	`case`	`case, group-by, agg-count`	`PASS; PASS`	`PASS`

WINDOW 分区与排序数据	`False`	`window-row-number`	`window-row-number`	`PASS; PASS`	`PASS`
CTE 基表约束传递	`False`	`where`	`where, join-on, cte`	`PASS; PASS`	`PASS`
递归 CTE 终止边界与沙盒熔断	`False`	`cte-recursive`	`union, cte-recursive, where`	`PASS; PASS`	`PASS`

WHERE 数值边界三态

- **策略说明**：针对数值谓词条件中的边界 c （如 `> c`、`<= c`），在数据行中强行注入临界值 $[c, c+1, c-1]$，分别覆盖均符合区 (T_{both})、临界差异区 (T_{diff}) 和均不符合区 ($T_{neither}$)。这能打破比较操作符（如 `>` 与 `>=`）在常规随机值下的假等价遮蔽，迫使边界逻辑错误显形。
- **Schema**: `course(course_id, title, dept_name, credits)`
- **标准答案 SQL**:

```
代码块
1   SELECT title FROM course WHERE credits > 3;
```

- **学生作答 SQL**:

```
代码块
1   SELECT title FROM course WHERE credits >= 3;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['where']`
- **策略检查结果**:
  a. `PASS` - course.credits values=[3, 4, 2, 7, 8, 1, 2, 1002], required=[2, 3, 4]
  b. `PASS` - expected KP=where, actual=['where']
- **动态生成的数据集**:

```
代码块
1   {
2     "course": [
3       {
4         "course_id": 5,
5         "title": "Marketing Lead",
6         "dept_name": "Physics",
7         "credits": 3
8       },
9       {
10        "course_id": 6,
11        "title": "Engineer",
12        "dept_name": "History",
13        "credits": 4
14      },
```

```
      {
        "course_id": 7,
        "title": "Analyst",
        "dept_name": "Comp. Sci.",
        "credits": 2
      },
      {
        "course_id": 8,
        "title": "Sales Manager",
        "dept_name": "Math",
        "credits": 7
      },
      {
        "course_id": 1,
        "title": "Marketing Lead",
        "dept_name": "Physics",
        "credits": 8
      },
      {
        "course_id": 2,
        "title": "Engineer",
        "dept_name": "History",
        "credits": 1
      },
      {
        "course_id": 3,
        "title": "Analyst",
        "dept_name": "Comp. Sci.",
        "credits": 2
      },
      {
        "course_id": 4,
        "title": "Sales Manager",
        "dept_name": "Math",
        "credits": 1002
      }
    ]
  }
```


- **标准输出样本**: `[('Engineer',), ('Sales Manager',), ('Marketing Lead',), ('Sales Manager',)]`
- **学生输出样本**: `[('Marketing Lead',), ('Engineer',), ('Sales Manager',), ('Marketing Lead',), ('Sales Manager',)]`

SELECT 投影列完整性

- **策略说明**：在数据生成阶段，根据 SQL 语法树仅对引用的列生成种子值限制行宽。当学生 SQL 漏投、多投或改写了投影字段（导致列名或列数不符）时，沙盒执行引擎的列结构验证机制（`columns_match`）将直接拦截并在 `select-basic`（投影缺失/错误）知识点上归因。
- **Schema**: `course(course_id, title, dept_name, credits)`
- **标准答案 SQL**:

代码块

```
SELECT title, credits FROM course WHERE credits > 3;
```

- **学生作答 SQL**:

```sql
SELECT title FROM course WHERE credits > 3;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['select-basic']`
- **策略检查结果**:
  a. `PASS` - course.credits values=[3, 4, 2, 7, 8, 1, 2, 1002], required=[2, 3, 4]
  b. `PASS` - standard_columns=['title', 'credits'], student_columns=['title']
  c. `PASS` - expected KP=select-basic, actual=['select-basic']
- **动态生成的数据集**:

代码块


```
{
  "course": [
    {
      "course_id": 5,
      "title": "Marketing Lead",
      "dept_name": "Physics",
      "credits": 3
    },
    {
      "course_id": 6,
      "title": "Engineer",
      "dept_name": "History",
      "credits": 4
    },
    {
      "course_id": 7,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 2
    },
    {
      "course_id": 8,
      "title": "Sales Manager",
      "dept_name": "Math",
      "credits": 7
    },
    {
      "course_id": 1,
      "title": "Marketing Lead",
      "dept_name": "Physics",
      "credits": 8
    },
    {
      "course_id": 2,
      "title": "Engineer",
      "dept_name": "History",
      "credits": 1
    },
    {
      "course_id": 3,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 2
    },
    {
```

```
        "course_id": 4,
        "title": "Sales Manager",
        "dept_name": "Math",
        "credits": 1002
      }
    ]
  }
```


- **标准输出样本**: `[('Engineer', 4), ('Sales Manager', 7), ('Marketing Lead', 8), ('Sales Manager', 1002)]`
- **学生输出样本**: `[('Engineer',), ('Sales Manager',), ('Marketing Lead',), ('Sales Manager',)]`

NULL 空值过滤探针

- **策略说明**：主动在某些数据行中注入 `None` (SQL 中的 `NULL`)，同时在其它行生成普通有效值。由于 SQL 采用三值逻辑，非标准的 `col = NULL` 比较永远返回 `Unknown` (即过滤后的空集)，而标准的 `col IS NULL` 能够匹配 `None` 行。因此，主动注入 `None` 能产生悬殊的执行结果差异。
- **Schema**: `student(ID, name, grade)`
- **标准答案 SQL**:

代码块

```
SELECT name FROM student WHERE grade IS NULL;
```

- **学生作答 SQL**:

代码块

```
SELECT name FROM student WHERE grade = NULL;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['comp-null', 'where']`
- **策略检查结果**:
  a. `PASS` - student.grade values=[None, 'B', 'C', None, 'A', 'B', 'C', None]
  b. `PASS` - expected KP=comp-null, actual=['comp-null', 'where']
- **动态生成的数据集**:

代码块


```
{
  "student": [
    {
      "ID": 7,
      "name": "Carol",
      "grade": null
    },
    {
      "ID": 8,
      "name": "Dave",
      "grade": "B"
    },
    {
      "ID": 1,
      "name": "Alice",
```

```
      "grade": "C"
    },
    {
      "ID": 2,
      "name": "Bob",
      "grade": null
    },
    {
      "ID": 3,
      "name": "Carol",
      "grade": "A"
    },
    {
      "ID": 4,
      "name": "Dave",
      "grade": "B"
    },
    {
      "ID": 5,
      "name": "Alice",
      "grade": "C"
    },
    {
      "ID": 6,
      "name": "Bob",
      "grade": null
    }
  ]
}
```


- **标准输出样本**: `[('Carol',), ('Bob',), ('Bob',)]`
- **学生输出样本**: `[]`

DISTINCT 去重探针

- **策略说明**：在满足数据表唯一性约束（排除 ID/SSN 等核心主键）的安全范围内，在 Row 0 和 Row 1 的非主键列上复制生成完全重复的数据行。当学生漏写 `DISTINCT` 去重修饰符时，学生 SQL 的执行结果将产生行数膨胀（包含重复行），与标准去重 SQL 产生行数分化。
- **Schema**: `takes(ID, course_id, sec_id, semester, year, grade)`
- **标准答案 SQL**:

代码块

```
SELECT DISTINCT course_id FROM takes;
```

- **学生作答 SQL**:

代码块

```
SELECT course_id FROM takes;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['distinct']`
- **策略检查结果**:
  a. `PASS` - takes.course_id duplicate_values=[2], values=[2, 2, 4, 5, 6, 7, 8, None]

b. `PASS` - expected KP=distinct, actual=['distinct']

- **动态生成的数据集**:

代码块

```
{
  "takes": [
    {
      "ID": 2,
      "course_id": 2,
      "sec_id": 8,
      "semester": "Summer",
      "year": 8,
      "grade": "B"
    },
    {
      "ID": 3,
      "course_id": 2,
      "sec_id": 8,
      "semester": "Winter",
      "year": 1,
      "grade": "C"
    },
    {
      "ID": 4,
      "course_id": 4,
      "sec_id": 2,
      "semester": "Fall",
      "year": 2,
      "grade": null
    },
    {
      "ID": 5,
      "course_id": 5,
      "sec_id": 3,
      "semester": "Spring",
      "year": 3,
      "grade": "A"
    },
    {
      "ID": 6,
      "course_id": 6,
      "sec_id": 4,
      "semester": "Summer",
      "year": 4,
      "grade": "B"
    },
    {
      "ID": 7,
      "course_id": 7,
      "sec_id": 5,
      "semester": "Winter",
      "year": 5,
      "grade": "C"
    },
    {
      "ID": 8,
      "course_id": 8,
      "sec_id": 6,
      "semester": "Fall",
      "year": 6,
```

```
        "grade": null
      },
      {
        "ID": null,
        "course_id": null,
        "sec_id": null,
        "semester": null,
        "year": null,
        "grade": null
      }
    ]
  }
```


- **标准输出样本**: `[(2,), (4,), (5,), (6,), (7,)]`
- **学生输出样本**: `[(2,), (2,), (4,), (5,), (6,)]`

JOIN 拓扑对齐与跨键漂移

- **策略说明**：使用多项式滚动哈希（Polynomial Hashing）对同组的 `table.column` 分配确定性且互不重合的偏移量（Shift），并在 Join Group 共享值池内进行动态碰撞排重。这能打乱同一行中多个外键列的值，防止因数据过于对称（如 `s_ID` 与 `i_ID` 相同）导致错连连接键（ON 条件）被同构屏蔽。
- **Schema**: `student(ID, name, dept_name); advisor(s_ID, i_ID)`
- **标准答案 SQL**:

代码块

```
SELECT student.name FROM student JOIN advisor ON student.ID = advisor.s_ID;
```

- **学生作答 SQL**:

代码块

```
SELECT student.name FROM student JOIN advisor ON student.ID = advisor.i_ID;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['join-on']`
- **策略检查结果**:
  a. `PASS` - student.ID=[1, 2, 3, 4, 5, 6, 7, 8], advisor.s_ID=[6, 7, 8, 1, 2, 3, 4], advisor.i_ID=[8, 1, 2, 3, 4, 5, 6]
  b. `PASS` - expected KP=join-on, actual=['join-on']
- **动态生成的数据集**:

代码块


```
{
  "student": [
    {
      "ID": 7,
      "name": "Carol",
      "dept_name": "Physics"
    },
    {
      "ID": 8,
      "name": "Dave",
      "dept_name": "History"
```

```
    },
    {
      "ID": 1,
      "name": "Alice",
      "dept_name": "Comp. Sci."
    },
    {
      "ID": 2,
      "name": "Bob",
      "dept_name": "Math"
    },
    {
      "ID": 3,
      "name": "Carol",
      "dept_name": "Physics"
    },
    {
      "ID": 4,
      "name": "Dave",
      "dept_name": "History"
    },
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci."
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math"
    }
  ],
  "advisor": [
    {
      "s_ID": 6,
      "i_ID": 8
    },
    {
      "s_ID": 7,
      "i_ID": 1
    },
    {
      "s_ID": 8,
      "i_ID": 2
    },
    {
      "s_ID": 1,
      "i_ID": 3
    },
    {
      "s_ID": 2,
      "i_ID": 4
    },
    {
      "s_ID": 3,
      "i_ID": 5
    },
    {
      "s_ID": 4,
      "i_ID": 6
    },
    {
```

```
        "s_ID": null,
        "i_ID": null
      }
    ]
  }
```

- **标准输出样本**: `[('Carol',), ('Dave',), ('Alice',), ('Bob',), ('Carol',)]`
- **学生输出样本**: `[('Dave',), ('Alice',), ('Bob',), ('Carol',), ('Dave',)]`

LEFT JOIN 悬浮元组

- **策略说明**：对关系子表（如 `takes`、`advisor`）的最后一行强制赋予 `None`，作为未匹配的“孤儿行”。这构建了天然的外连接悬浮元组，使得 `LEFT JOIN`（保留该孤儿行并填充 NULL）与 `INNER JOIN`（剔除该行）产生行数和空值项差异。
- **Schema**: `student(ID, name, dept_name); takes(ID, course_id)`
- **标准答案 SQL**:

代码块

```
SELECT student.name, takes.course_id FROM student LEFT JOIN takes ON student.ID = takes.ID;
```

- **学生作答 SQL**:

代码块

```
SELECT student.name, takes.course_id FROM student INNER JOIN takes ON student.ID = takes.ID;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['join-left', 'join-inner']`
- **策略检查结果**:
  a. `PASS` - takes.ID values=[2, 3, 4, 5, 6, 7, 8, None], evidence={'sandbox_executed': True, 'student_exec_ok': True, 'student_exec_error': None, 'is_equivalent_on_generated_data': False, 'ordered_compare': False, 'row_count_match': False, 'standard_row_count': 8, 'student_row_count': 7, 'columns_match': True, 'standard_columns': ['name', 'course_id'], 'student_columns': ['name', 'course_id'], 'standard_duplicate_row_count': 0, 'student_duplicate_row_count': 0, 'suspected_cartesian_product': False, 'only_in_standard_sample': [('Alice', None)], 'only_in_student_sample': [], 'standard_sample_rows': [('Carol', 7), ('Dave', 8), ('Alice', None), ('Bob', 2), ('Carol', 3)], 'student_sample_rows': [('Carol', 7), ('Dave', 8), ('Bob', 2), ('Carol', 3), ('Dave', 4)]}
  b. `PASS` - expected KP=join-left, actual=['join-left', 'join-inner']
- **动态生成的数据集**:

代码块

```
{
  "student": [
    {
      "ID": 7,
      "name": "Carol",
      "dept_name": "Physics"
    },
    {
      "ID": 8,
      "name": "Dave",
      "dept_name": "History"
```

```
    },
    {
      "ID": 1,
      "name": "Alice",
      "dept_name": "Comp. Sci."
    },
    {
      "ID": 2,
      "name": "Bob",
      "dept_name": "Math"
    },
    {
      "ID": 3,
      "name": "Carol",
      "dept_name": "Physics"
    },
    {
      "ID": 4,
      "name": "Dave",
      "dept_name": "History"
    },
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci."
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math"
    }
  ],
  "takes": [
    {
      "ID": 2,
      "course_id": 2
    },
    {
      "ID": 3,
      "course_id": 3
    },
    {
      "ID": 4,
      "course_id": 4
    },
    {
      "ID": 5,
      "course_id": 5
    },
    {
      "ID": 6,
      "course_id": 6
    },
    {
      "ID": 7,
      "course_id": 7
    },
    {
      "ID": 8,
      "course_id": 8
    },
    {
```

```
        "ID": null,
        "course_id": null
      }
    ]
  }
```

- **标准输出样本**: `[('Carol', 7), ('Dave', 8), ('Alice', None), ('Bob', 2), ('Carol', 3)]`
- **学生输出样本**: `[('Carol', 7), ('Dave', 8), ('Bob', 2), ('Carol', 3), ('Dave', 4)]`

GROUP BY 分组粒度错

- **策略说明**：系统对每张表默认生成 4~8 行，并在分组列上填充多个不同的异构分类键。这能保证当学生把分组字段写错（例如按 `building` 错写为按 `dept_name` 分组）时，各组 of 聚合与累加组合必然发生错位，导致求和或计数数组与标答不等价。
- **Schema**: `instructor(ID, name, dept_name, salary, building)`
- **标准答案 SQL**:

代码块

```
SELECT SUM(salary) FROM instructor GROUP BY dept_name;
```

- **学生作答 SQL**:

代码块

```
SELECT SUM(salary) FROM instructor GROUP BY building;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['group-by']`
- **策略检查结果**:
  a. `PASS` - expected KP=group-by, actual=['group-by']
- **动态生成的数据集**:

代码块

```
{
  "instructor": [
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 4,
      "building": "building_6"
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 5,
      "building": "building_7"
    },
    {
      "ID": 7,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 6,
```

```
      "building": "building_8"
    },
    {
      "ID": 8,
      "name": "Dave",
      "dept_name": "History",
      "salary": 7,
      "building": "building_1"
    },
    {
      "ID": 1,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 8,
      "building": "building_2"
    },
    {
      "ID": 2,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 1,
      "building": "building_3"
    },
    {
      "ID": 3,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 2,
      "building": "building_4"
    },
    {
      "ID": 4,
      "name": "Dave",
      "dept_name": "History",
      "salary": 3,
      "building": "building_5"
    }
  ]
}
```


- **标准输出样本**: `[(12,), (10,), (6,), (8,)]`
- **学生输出样本**: `[(7,), (8,), (1,), (2,), (3,)]`

HAVING SUM 聚合边界三态

- **策略说明**：由于 HAVING 过滤发生在分组聚合之后，不能直接改写基表单行数据。系统将记录按分组归类，并分别对各组数据做三态控制，使各分组的聚合 `SUM` 目标值精确达到 $c+1$（阳性通过）、c（临界差异）和 $c-1$（阴性过滤）。再除以组内行数 k 填充回单行记录中，激活 HAVING 谓词边界过滤。
- **Schema**: `instructor(ID, name, dept_name, salary)`
- **标准答案 SQL**:

代码块

```
SELECT dept_name FROM instructor GROUP BY dept_name HAVING SUM(salary) > 80000;
```

- **学生作答 SQL**:

```sql
SELECT dept_name FROM instructor GROUP BY dept_name HAVING SUM(salary) < 80000;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['having', 'group-by']`
- **策略检查结果**:
  a. `PASS` - SUM metrics={'Comp. Sci.': 80001.0, 'Math': 79999.0, 'Physics': 80000.0, 'History': 8000.0}, boundary=80000
  b. `PASS` - expected KP=having, actual=['having', 'group-by']
- **动态生成的数据集**:

代码块

```json
{
  "instructor": [
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 40000.5
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 39999.5
    },
    {
      "ID": 7,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 40000.0
    },
    {
      "ID": 8,
      "name": "Dave",
      "dept_name": "History",
      "salary": 4000.0
    },
    {
      "ID": 1,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 40000.5
    },
    {
      "ID": 2,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 39999.5
    },
    {
      "ID": 3,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 40000.0
    },
    {
      "ID": 4,
      "name": "Dave",
```

```
        "dept_name": "History",
        "salary": 4000.0
      }
    ]
  }
```


- **标准输出样本**: `[('Comp. Sci.',)]`
- **学生输出样本**: `[('History',), ('Math',)]`

HAVING AVG 聚合边界三态

- **策略说明**：同 SUM 策略。控制每个分组的 `AVG` （均值）结果值，使其分别精确达到 $[c+1, c, c-1]$。数据生成时，直接使该分组内所有行的数值列均等于对应的目标值，使其平均值精确被控，引爆 HAVING 边界判断差异。
- **Schema**: `instructor(ID, name, dept_name, salary)`
- **标准答案 SQL**:

代码块

```
SELECT dept_name FROM instructor GROUP BY dept_name HAVING AVG(salary) > 50000;
```

- **学生作答 SQL**:

代码块

```
SELECT dept_name FROM instructor GROUP BY dept_name HAVING AVG(salary) < 50000;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['having', 'group-by']`
- **策略检查结果**:
  a. `PASS` - AVG metrics={'Comp. Sci.': 50001.0, 'Math': 49999.0, 'Physics': 50000.0, 'History': 5000.0}, boundary=50000
  b. `PASS` - expected KP=having, actual=['having', 'group-by']
- **动态生成的数据集**:

代码块


```
{
  "instructor": [
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 50001
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 49999
    },
    {
      "ID": 7,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 50000
```

```
    },
    {
      "ID": 8,
      "name": "Dave",
      "dept_name": "History",
      "salary": 5000.0
    },
    {
      "ID": 1,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 50001
    },
    {
      "ID": 2,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 49999
    },
    {
      "ID": 3,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 50000
    },
    {
      "ID": 4,
      "name": "Dave",
      "dept_name": "History",
      "salary": 5000.0
    }
  ]
}
```


- **标准输出样本**: `[('Comp. Sci.',)]`
- **学生输出样本**: `[('History',), ('Math',)]`

HAVING MIN 聚合边界三态

- **策略说明**：控制每个分组的 `MIN`（极小值）结果值，使其分别达到 $[c+1, c, c-1]$。数据干预时，将该组 Row 0 设为目标值 `T`，组内其余行均设为 `T + 1`，保证该组的最小值精确锁定在 `T`，校验极值过滤逻辑。
- **Schema**: `instructor(ID, name, dept_name, salary)`
- **标准答案 SQL**:

代码块

```
SELECT dept_name FROM instructor GROUP BY dept_name HAVING MIN(salary) > 30000;
```

- **学生作答 SQL**:

代码块

```
SELECT dept_name FROM instructor GROUP BY dept_name HAVING MIN(salary) < 30000;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['having', 'group-by']`

- **策略检查结果**:
  a. `PASS` - MIN metrics={'Comp. Sci.': 30001, 'Math': 29999, 'Physics': 30000, 'History': 3000.0}, boundary=30000
  b. `PASS` - expected KP=having, actual=['having', 'group-by']
- **动态生成的数据集**:

代码块


```
{
  "instructor": [
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 30001
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 29999
    },
    {
      "ID": 7,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 30000
    },
    {
      "ID": 8,
      "name": "Dave",
      "dept_name": "History",
      "salary": 3000.0
    },
    {
      "ID": 1,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 30002
    },
    {
      "ID": 2,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 30000
    },
    {
      "ID": 3,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 30001
    },
    {
      "ID": 4,
      "name": "Dave",
      "dept_name": "History",
      "salary": 3001.0
    }
  ]
}
```

- **标准输出样本**: `[('Comp. Sci.',)]`
- **学生输出样本**: `[('History',), ('Math',)]`

HAVING MAX 聚合边界三态

- **策略说明**：控制每个分组的 `MAX`（极大值）结果值，使其分别达到 $[c+1, c, c-1]$。数据干预时，将该组 Row 0 设为目标值 `T`，组内其余行均设为 `T - 1`，保证该组的最大值精确锁定在 `T`，校验极值过滤逻辑。
- **Schema**: `instructor(ID, name, dept_name, salary)`
- **标准答案 SQL**:

```
代码块
1   SELECT dept_name FROM instructor GROUP BY dept_name HAVING MAX(salary) > 90000;
```

- **学生作答 SQL**:

```
代码块
1   SELECT dept_name FROM instructor GROUP BY dept_name HAVING MAX(salary) < 90000;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['having', 'group-by']`
- **策略检查结果**:
  a. `PASS` - MAX metrics={'Comp. Sci.': 90001, 'Math': 89999, 'Physics': 90000, 'History': 9000.0}, boundary=90000
  b. `PASS` - expected KP=having, actual=['having', 'group-by']
- **动态生成的数据集**:

```
代码块
1   {
2     "instructor": [
3       {
4         "ID": 5,
5         "name": "Alice",
6         "dept_name": "Comp. Sci.",
7         "salary": 90001
8       },
9       {
10        "ID": 6,
11        "name": "Bob",
12        "dept_name": "Math",
13        "salary": 89999
14      },
15      {
16        "ID": 7,
17        "name": "Carol",
18        "dept_name": "Physics",
19        "salary": 90000
20      },
21      {
22        "ID": 8,
23        "name": "Dave",
24        "dept_name": "History",
25        "salary": 9000.0
26      },
```

```
      {
        "ID": 1,
        "name": "Alice",
        "dept_name": "Comp. Sci.",
        "salary": 90000
      },
      {
        "ID": 2,
        "name": "Bob",
        "dept_name": "Math",
        "salary": 89998
      },
      {
        "ID": 3,
        "name": "Carol",
        "dept_name": "Physics",
        "salary": 89999
      },
      {
        "ID": 4,
        "name": "Dave",
        "dept_name": "History",
        "salary": 8999.0
      }
    ]
  }
```


- **标准输出样本**: `[('Comp. Sci.',)]`
- **学生输出样本**: `[('History',), ('Math',)]`

HAVING COUNT 组大小三态

- **策略说明**：对于行记录数限制（`COUNT`），无法通过改写数值列生效。本策略直接重排和限制分组列的物理键分配，使得各分组的物理行数（即组大小）分别等于 $[c+1, c, c-1]$，从而当比较行数写错时直接被沙盒过滤拦截。
- **Schema**: `instructor(ID, name, dept_name, salary)`
- **标准答案 SQL**:

代码块

```
SELECT dept_name FROM instructor GROUP BY dept_name HAVING COUNT(ID) >= 2;
```

- **学生作答 SQL**:

代码块

```
SELECT dept_name FROM instructor GROUP BY dept_name HAVING COUNT(ID) > 2;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['having', 'group-by']`
- **策略检查结果**:
  a. `PASS` - COUNT metrics={'Comp. Sci.': 3, 'Math': 2, 'Physics': 1, 'History': 2}, boundary=2
  b. `PASS` - expected KP=having, actual=['having', 'group-by']
- **动态生成的数据集**:

代码块

```
  "instructor": [
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 4
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Comp. Sci.",
      "salary": 5
    },
    {
      "ID": 7,
      "name": "Carol",
      "dept_name": "Comp. Sci.",
      "salary": 6
    },
    {
      "ID": 8,
      "name": "Dave",
      "dept_name": "Math",
      "salary": 7
    },
    {
      "ID": 1,
      "name": "Alice",
      "dept_name": "Math",
      "salary": 8
    },
    {
      "ID": 2,
      "name": "Bob",
      "dept_name": "Physics",
      "salary": 1
    },
    {
      "ID": 3,
      "name": "Carol",
      "dept_name": "History",
      "salary": 2
    },
    {
      "ID": 4,
      "name": "Dave",
      "dept_name": "History",
      "salary": 3
    }
  ]
}
```

- **标准输出样本**: `[('Comp. Sci.',), ('History',), ('Math',)]`
- **学生输出样本**: `[('Comp. Sci.',)]`

ORDER BY 有序精确比对

- **策略说明**：提取排序关键字并生成具有单调递增/递减特征的数据序列。一旦检测到 SQL 中包含 `ORDER BY`，沙盒结果比对模块将关闭无序的频次比对（Counter），转为严格的顺序列表比对（`std_rows == stu_rows`），使排序方向或排序列写错直接暴露。
- **Schema**: `course(course_id, title, dept_name, credits)`
- **标准答案 SQL**:

代码块

```
SELECT title FROM course ORDER BY credits DESC;
```

- **学生作答 SQL**:

代码块

```
SELECT title FROM course ORDER BY credits ASC;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['order-by']`
- **策略检查结果**:
  a. `PASS` - evidence={'sandbox_executed': True, 'student_exec_ok': True, 'student_exec_error': None, 'is_equivalent_on_generated_data': False, 'ordered_compare': True, 'row_count_match': True, 'standard_row_count': 8, 'student_row_count': 8, 'columns_match': True, 'standard_columns': ['title'], 'student_columns': ['title'], 'standard_duplicate_row_count': 4, 'student_duplicate_row_count': 4, 'suspected_cartesian_product': False, 'only_in_standard_sample': [], 'only_in_student_sample': [], 'standard_sample_rows': [('Marketing Lead',), ('Sales Manager',), ('Analyst',), ('Engineer',), ('Marketing Lead',)], 'student_sample_rows': [('Engineer',), ('Analyst',), ('Sales Manager',), ('Marketing Lead',), ('Engineer',)]}
  b. `PASS` - expected KP=order-by, actual=['order-by']
- **动态生成的数据集**:

代码块

```
{
  "course": [
    {
      "course_id": 5,
      "title": "Marketing Lead",
      "dept_name": "Physics",
      "credits": 4
    },
    {
      "course_id": 6,
      "title": "Engineer",
      "dept_name": "History",
      "credits": 5
    },
    {
      "course_id": 7,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 6
    },
    {
      "course_id": 8,
      "title": "Sales Manager",
```

```
        "dept_name": "Math",
        "credits": 7
      },
      {
        "course_id": 1,
        "title": "Marketing Lead",
        "dept_name": "Physics",
        "credits": 8
      },
      {
        "course_id": 2,
        "title": "Engineer",
        "dept_name": "History",
        "credits": 1
      },
      {
        "course_id": 3,
        "title": "Analyst",
        "dept_name": "Comp. Sci.",
        "credits": 2
      },
      {
        "course_id": 4,
        "title": "Sales Manager",
        "dept_name": "Math",
        "credits": 3
      }
    ]
  }
```


- **标准输出样本**: `[('Marketing Lead',), ('Sales Manager',), ('Analyst',), ('Engineer',), ('Marketing Lead',)]`
- **学生输出样本**: `[('Engineer',), ('Analyst',), ('Sales Manager',), ('Marketing Lead',), ('Engineer',)]`

LIMIT 行数边界

- **策略说明**：限制输出的元组行数。通过沙盒直接验证 `LIMIT` 或 `OFFSET` 参数的数值偏差，让多取或少取数据的学生 SQL 产生行数不等。
- **Schema**: `course(course_id, title, dept_name, credits)`
- **标准答案 SQL**:

代码块

```
SELECT title FROM course LIMIT 3;
```

- **学生作答 SQL**:

代码块

```
SELECT title FROM course LIMIT 5;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['limit']`
- **策略检查结果**:

a. `PASS` - standard/student=3/5

b. `PASS` - expected KP=limit, actual=['limit']

- **动态生成的数据集**:

代码块

```
{
  "course": [
    {
      "course_id": 5,
      "title": "Marketing Lead",
      "dept_name": "Physics",
      "credits": 4
    },
    {
      "course_id": 6,
      "title": "Engineer",
      "dept_name": "History",
      "credits": 5
    },
    {
      "course_id": 7,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 6
    },
    {
      "course_id": 8,
      "title": "Sales Manager",
      "dept_name": "Math",
      "credits": 7
    },
    {
      "course_id": 1,
      "title": "Marketing Lead",
      "dept_name": "Physics",
      "credits": 8
    },
    {
      "course_id": 2,
      "title": "Engineer",
      "dept_name": "History",
      "credits": 1
    },
    {
      "course_id": 3,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 2
    },
    {
      "course_id": 4,
      "title": "Sales Manager",
      "dept_name": "Math",
      "credits": 3
    }
  ]
}
```

- **标准输出样本**: `[('Marketing Lead',), ('Engineer',), ('Analyst',)]`
- **学生输出样本**: `[('Marketing Lead',), ('Engineer',), ('Analyst',), ('Sales Manager',), ('Marketing Lead',)]`

子查询内外层值域重合

- **策略说明**：提取子查询中的关联列，在父子表之间建立主外键或数据范围的重合（对齐 ID 共享值池，打通拓扑通路）。再配合子查询内部的过滤谓词，强行在子表中构造阳性重合行（满足过滤）、阴性混淆行（不满足过滤）和悬浮行，触发子查询的过滤选择权，暴露内外层值域逻辑错。
- **Schema**: `student(ID, name, dept_name); takes(ID, course_id, year)`
- **标准答案 SQL**:

```
代码块
1   SELECT name FROM student WHERE ID IN (SELECT ID FROM takes WHERE year = 2017);
```

- **学生作答 SQL**:

```
代码块
1   SELECT name FROM student WHERE ID NOT IN (SELECT ID FROM takes WHERE year = 2017);
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['where']`
- **策略检查结果**:
  a. `PASS` - student.ID=[1, 2, 3, 4, 5, 6, 7, 8], takes.ID=[2, 3, 4, 5, 6, 7, 8, None]
  b. `PASS` - expected KP=where, actual=['where']
- **动态生成的数据集**:

```
代码块
1   {
2     "student": [
3       {
4         "ID": 7,
5         "name": "Carol",
6         "dept_name": "Physics"
7       },
8       {
9         "ID": 8,
10        "name": "Dave",
11        "dept_name": "History"
12      },
13      {
14        "ID": 1,
15        "name": "Alice",
16        "dept_name": "Comp. Sci."
17      },
18      {
19        "ID": 2,
20        "name": "Bob",
21        "dept_name": "Math"
22      },
23      {
24        "ID": 3,
```

```
      "name": "Carol",
      "dept_name": "Physics"
    },
    {
      "ID": 4,
      "name": "Dave",
      "dept_name": "History"
    },
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci."
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math"
    }
  ],
  "takes": [
    {
      "ID": 2,
      "course_id": 2,
      "year": 2017
    },
    {
      "ID": 3,
      "course_id": 3,
      "year": 2018
    },
    {
      "ID": 4,
      "course_id": 4,
      "year": 2016
    },
    {
      "ID": 5,
      "course_id": 5,
      "year": 3
    },
    {
      "ID": 6,
      "course_id": 6,
      "year": 4
    },
    {
      "ID": 7,
      "course_id": 7,
      "year": 5
    },
    {
      "ID": 8,
      "course_id": 8,
      "year": 6
    },
    {
      "ID": null,
      "course_id": null,
      "year": 3016
    }
  ]
```

```
86  }
```

- **标准输出样本**: `[('Bob',)]`
- **学生输出样本**: `[('Carol',), ('Dave',), ('Alice',), ('Carol',), ('Dave',)]`

相关子查询内外层关联

- **策略说明**：静态扫描子查询的过滤条件，识别并提取外层主表的引用列（如 `t.ID = s.ID`）。在生成数据时，确保被引用的主表 ID 与子查询中关联表的 ID 发生数据交叉（在内层表生成对应相关变量的多态数据），以便在子查询被多次关联扫描时，逻辑漏洞能被沙盒识别。
- **Schema**: `student(ID, name, dept_name); takes(ID, course_id, year)`
- **标准答案 SQL**:

代码块

```
1  SELECT name FROM student s WHERE EXISTS (SELECT 1 FROM takes t WHERE t.ID = s.ID AND t.year = 2017);
```

- **学生作答 SQL**:

代码块

```
1  SELECT name FROM student s WHERE EXISTS (SELECT 1 FROM takes t WHERE t.ID = s.ID AND t.year = 2018);
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['where', 'subquery-correlated']`
- **策略检查结果**:
  a. `PASS` - student.ID=[1, 2, 3, 4, 5, 6, 7, 8], takes.ID=[2, 3, 4, 5, 6, 7, 8, None]
  b. `PASS` - takes.year values=[2017, 2018, 2016, 2018, 2017, 2018, 2017, 3016], required=[2016, 2017, 2018]
  c. `PASS` - expected KP=subquery-correlated, actual=['where', 'subquery-correlated']
- **动态生成的数据集**:

代码块

```
1  {
2    "student": [
3      {
4        "ID": 7,
5        "name": "Carol",
6        "dept_name": "Physics"
7      },
8      {
9        "ID": 8,
10       "name": "Dave",
11       "dept_name": "History"
12     },
13     {
14       "ID": 1,
15       "name": "Alice",
16       "dept_name": "Comp. Sci."
17     },
18     {
19       "ID": 2,
20       "name": "Bob",
21       "dept_name": "Math"
```

```
    },
    {
      "ID": 3,
      "name": "Carol",
      "dept_name": "Physics"
    },
    {
      "ID": 4,
      "name": "Dave",
      "dept_name": "History"
    },
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci."
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math"
    }
  ],
  "takes": [
    {
      "ID": 2,
      "course_id": 2,
      "year": 2017
    },
    {
      "ID": 3,
      "course_id": 3,
      "year": 2018
    },
    {
      "ID": 4,
      "course_id": 4,
      "year": 2016
    },
    {
      "ID": 5,
      "course_id": 5,
      "year": 2018
    },
    {
      "ID": 6,
      "course_id": 6,
      "year": 2017
    },
    {
      "ID": 7,
      "course_id": 7,
      "year": 2018
    },
    {
      "ID": 8,
      "course_id": 8,
      "year": 2017
    },
    {
      "ID": null,
      "course_id": null,
      "year": 3016
```

```
84        }
85      ]
86    }
```

- **标准输出样本**: `[('Dave',), ('Bob',), ('Bob',)]`
- **学生输出样本**: `[('Carol',), ('Carol',), ('Alice',)]`

集合操作 UNION 去重差异

- **策略说明**：在集合算子（`UNION` / `UNION ALL`）左右两侧 of 子查询结果中生成完全重复的行。当学生混淆 `UNION`（集合自动去重）与 `UNION ALL`（保留所有重复行）时，学生 SQL 的执行输出将包含额外的重复行，行数随之分化。
- **Schema**: `course(course_id, title, dept_name, credits)`
- **标准答案 SQL**:

代码块

```
SELECT title FROM course WHERE dept_name = 'Math' UNION SELECT title FROM course WHERE dept_name =
'Physics';
```

- **学生作答 SQL**:

代码块

```
SELECT title FROM course WHERE dept_name = 'Math' UNION ALL SELECT title FROM course WHERE dept_name =
'Physics';
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['union']`
- **策略检查结果**:
  a. `PASS` - expected KP=union, actual=['union']
- **动态生成的数据集**:

代码块

```
{
  "course": [
    {
      "course_id": 5,
      "title": "Marketing Lead",
      "dept_name": "Math",
      "credits": 4
    },
    {
      "course_id": 6,
      "title": "Engineer",
      "dept_name": "Physics",
      "credits": 5
    },
    {
      "course_id": 7,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 6
    },
    {
```

```
        "course_id": 8,
        "title": "Sales Manager",
        "dept_name": "Math",
        "credits": 7
      },
      {
        "course_id": 1,
        "title": "Marketing Lead",
        "dept_name": "Physics",
        "credits": 8
      },
      {
        "course_id": 2,
        "title": "Engineer",
        "dept_name": "History",
        "credits": 1
      },
      {
        "course_id": 3,
        "title": "Analyst",
        "dept_name": "Comp. Sci.",
        "credits": 2
      },
      {
        "course_id": 4,
        "title": "Sales Manager",
        "dept_name": "not_Math",
        "credits": 3
      }
    ]
  }
```


- **标准输出样本**: `[('Engineer',), ('Marketing Lead',), ('Sales Manager',)]`
- **学生输出样本**: `[('Marketing Lead',), ('Sales Manager',), ('Engineer',), ('Marketing Lead',)]`

集合操作 INTERSECT 交集差异

- **策略说明**：在数据生成阶段，分别生成“仅满足左侧条件”、“仅满足右侧条件”以及“同时满足两侧条件”的记录。当学生错写集合操作符（如用 `UNION` 替代了 `INTERSECT`）时，沙盒执行结果将从交集空集或子集膨胀为并集，暴露逻辑错。
- **Schema**: `course(course_id, title, dept_name, credits)`
- **标准答案 SQL**:

代码块

```
SELECT title FROM course WHERE dept_name = 'Math' INTERSECT SELECT title FROM course WHERE credits > 3;
```

- **学生作答 SQL**:

代码块

```
SELECT title FROM course WHERE dept_name = 'Math' UNION SELECT title FROM course WHERE credits > 3;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['intersect']`
- **策略检查结果**:

a. `PASS` - expected KP=intersect, actual=['intersect']

- **动态生成的数据集**:

代码块

```
{
  "course": [
    {
      "course_id": 5,
      "title": "Marketing Lead",
      "dept_name": "Math",
      "credits": 3
    },
    {
      "course_id": 6,
      "title": "Engineer",
      "dept_name": "History",
      "credits": 4
    },
    {
      "course_id": 7,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 2
    },
    {
      "course_id": 8,
      "title": "Sales Manager",
      "dept_name": "Math",
      "credits": 7
    },
    {
      "course_id": 1,
      "title": "Marketing Lead",
      "dept_name": "Physics",
      "credits": 8
    },
    {
      "course_id": 2,
      "title": "Engineer",
      "dept_name": "History",
      "credits": 1
    },
    {
      "course_id": 3,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 2
    },
    {
      "course_id": 4,
      "title": "Sales Manager",
      "dept_name": "not_Math",
      "credits": 1002
    }
  ]
}
```

- **标准输出样本**: `[('Marketing Lead',), ('Sales Manager',)]`

- **学生输出样本**: `[('Engineer',), ('Marketing Lead',), ('Sales Manager',)]`

集合操作 EXCEPT 排他差异

- **策略说明**：提取 EXCEPT 右侧的过滤条件并在数据中生成排他数据行。这能保证当学生漏写了 `EXCEPT` 差集排除逻辑时，学生 SQL 的输出中会多出本应该被剔除的行，打破等价性。
- **Schema**: `course(course_id, title, dept_name, credits)`
- **标准答案 SQL**:

代码块

```
SELECT title FROM course EXCEPT SELECT title FROM course WHERE dept_name = 'Physics';
```

- **学生作答 SQL**:

代码块

```
SELECT title FROM course;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['except', 'where']`
- **策略检查结果**:
  a. `PASS` - expected KP=except, actual=['except', 'where']
- **动态生成的数据集**:

代码块

```
{
  "course": [
    {
      "course_id": 5,
      "title": "Marketing Lead",
      "dept_name": "Physics",
      "credits": 4
    },
    {
      "course_id": 6,
      "title": "Engineer",
      "dept_name": "History",
      "credits": 5
    },
    {
      "course_id": 7,
      "title": "Analyst",
      "dept_name": "Comp. Sci.",
      "credits": 6
    },
    {
      "course_id": 8,
      "title": "Sales Manager",
      "dept_name": "Math",
      "credits": 7
    },
    {
      "course_id": 1,
      "title": "Marketing Lead",
```

```
        "dept_name": "Physics",
        "credits": 8
      },
      {
        "course_id": 2,
        "title": "Engineer",
        "dept_name": "History",
        "credits": 1
      },
      {
        "course_id": 3,
        "title": "Analyst",
        "dept_name": "Comp. Sci.",
        "credits": 2
      },
      {
        "course_id": 4,
        "title": "Sales Manager",
        "dept_name": "not_Physics",
        "credits": 3
      }
    ]
  }
```


- **标准输出样本**: `[('Analyst',), ('Engineer',), ('Sales Manager',)]`
- **学生输出样本**: `[('Marketing Lead',), ('Engineer',), ('Analyst',), ('Sales Manager',), ('Marketing Lead',)]`

CASE WHEN 分支边界三态

- **策略说明**：针对 CASE WHEN 块中的各个分支条件（如 `amount > 100`），分别产生满足三态边界（c 、$c+1$、$c-1$）的测试数据，从而在沙盒执行时强制遍历所有计算和转换分支，校验条件边界的准确性。
- **Schema**: `sales(sale_id, category, amount)`
- **标准答案 SQL**:

代码块

```
SELECT category, SUM(CASE WHEN amount > 100 THEN amount ELSE 0 END) AS big_sales FROM sales GROUP BY category;
```

- **学生作答 SQL**:

代码块

```
SELECT category, SUM(CASE WHEN amount >= 100 THEN amount ELSE 0 END) AS big_sales FROM sales GROUP BY category;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['case', 'group-by', 'agg-count']`
- **策略检查结果**:
  a. `PASS` - sales.amount values=[100, 101, 99, 6, 7, 8, 1, 1099], required=[99, 100, 101]
  b. `PASS` - expected KP=case, actual=['case', 'group-by', 'agg-count']
- **动态生成的数据集**:

```
{
  "sales": [
    {
      "sale_id": 6,
      "category": "category_1",
      "amount": 100
    },
    {
      "sale_id": 7,
      "category": "category_2",
      "amount": 101
    },
    {
      "sale_id": 8,
      "category": "category_3",
      "amount": 99
    },
    {
      "sale_id": 1,
      "category": "category_4",
      "amount": 6
    },
    {
      "sale_id": 2,
      "category": "category_5",
      "amount": 7
    },
    {
      "sale_id": 3,
      "category": "category_6",
      "amount": 8
    },
    {
      "sale_id": 4,
      "category": "category_7",
      "amount": 1
    },
    {
      "sale_id": 5,
      "category": "category_8",
      "amount": 1099
    }
  ]
}
```


- **标准输出样本**: `[('category_1', 0), ('category_2', 101), ('category_3', 0), ('category_4', 0), ('category_5', 0)]`
- **学生输出样本**: `[('category_1', 100), ('category_2', 101), ('category_3', 0), ('category_4', 0), ('category_5', 0)]`

WINDOW 分区与排序数据

- **策略说明**：提取窗口函数的排序列与分区列，在数据行中产生乱序值和重复的分组值。如果学生在 `OVER` 子句中遗漏了 `PARTITION BY`，排序编号会出现全局自增而非分区独立重置的特征，导致数据不一致。
- **Schema**: `instructor(ID, name, dept_name, salary)`
- **标准答案 SQL**:

代码块

```sql
SELECT name, ROW_NUMBER() OVER (PARTITION BY dept_name ORDER BY salary DESC) AS rank FROM instructor;
```

- **学生作答 SQL**:

代码块

```sql
SELECT name, ROW_NUMBER() OVER (ORDER BY salary DESC) AS rank FROM instructor;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['window-row-number']`
- **策略检查结果**:
  a. `PASS` - dept group_counts={'Comp. Sci.': 2, 'Math': 2, 'Physics': 2, 'History': 2}, salaries=[4, 5, 6, 7, 8, 1, 2, 3]
  b. `PASS` - expected KP=window-row-number, actual=['window-row-number']
- **动态生成的数据集**:

代码块

```json
{
  "instructor": [
    {
      "ID": 5,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 4
    },
    {
      "ID": 6,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 5
    },
    {
      "ID": 7,
      "name": "Carol",
      "dept_name": "Physics",
      "salary": 6
    },
    {
      "ID": 8,
      "name": "Dave",
      "dept_name": "History",
      "salary": 7
    },
    {
      "ID": 1,
      "name": "Alice",
      "dept_name": "Comp. Sci.",
      "salary": 8
    },
    {
      "ID": 2,
      "name": "Bob",
      "dept_name": "Math",
      "salary": 1
    },
    {
      "ID": 3,
```

```
        "name": "Carol",
        "dept_name": "Physics",
        "salary": 2
      },
      {
        "ID": 4,
        "name": "Dave",
        "dept_name": "History",
        "salary": 3
      }
    ]
  }
```


- **标准输出样本**: `[('Alice', 1), ('Alice', 2), ('Dave', 1), ('Dave', 2), ('Bob', 1)]`
- **学生输出样本**: `[('Alice', 1), ('Dave', 2), ('Carol', 3), ('Bob', 4), ('Alice', 5)]`

CTE 基表约束传递

- **策略说明**：回溯 CTE（WITH 表达式）内部引用的底层基表并针对这些基表进行三态造数，而拒绝直接预造 CTE 临时表。CTE 定义与外层 `JOIN` 均交给 SQLite 原生执行，确保基表约束能自然传导至最外层，校验 CTE 基表约束传递性。
- **Schema**: `works(company_name, person_name, salary); company(company_name, city)`
- **标准答案 SQL**:

代码块

```
WITH big_co AS (SELECT company_name FROM company WHERE city = 'Beijing') SELECT person_name, salary FROM works JOIN big_co ON works.company_name = big_co.company_name WHERE salary > 10000;
```

- **学生作答 SQL**:

代码块

```
WITH big_co AS (SELECT company_name FROM company WHERE city = 'Beijing') SELECT person_name, salary FROM works JOIN big_co ON works.company_name = big_co.company_name WHERE salary < 10000;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['where', 'join-on', 'cte']`
- **策略检查结果**:
  a. `PASS` - works.salary values=[10000, 10001, 9999, 6, 7, 8, 1, 10999], required=[9999, 10000, 10001]
  b. `PASS` - expected KP=where, actual=['where', 'join-on', 'cte']
- **动态生成的数据集**:

代码块

```
{
  "works": [
    {
      "company_name": "Bob",
      "person_name": "Bob",
      "salary": 10000
    },
    {
      "company_name": "Carol",
      "person_name": "Carol",
      "salary": 10001
```

```
    },
    {
      "company_name": "Dave",
      "person_name": "Dave",
      "salary": 9999
    },
    {
      "company_name": "Alice",
      "person_name": "Alice",
      "salary": 6
    },
    {
      "company_name": "Bob",
      "person_name": "Bob",
      "salary": 7
    },
    {
      "company_name": "Carol",
      "person_name": "Carol",
      "salary": 8
    },
    {
      "company_name": "Dave",
      "person_name": "Dave",
      "salary": 1
    },
    {
      "company_name": null,
      "person_name": null,
      "salary": 10999
    }
  ],
  "company": [
    {
      "company_name": "Carol",
      "city": "Beijing"
    },
    {
      "company_name": "Dave",
      "city": "city_2"
    },
    {
      "company_name": "Alice",
      "city": "city_3"
    },
    {
      "company_name": "Bob",
      "city": "city_4"
    },
    {
      "company_name": "Carol",
      "city": "city_5"
    },
    {
      "company_name": "Dave",
      "city": "city_6"
    },
    {
      "company_name": "Alice",
      "city": "city_7"
    },
    {
```

```
        "company_name": "Bob",
        "city": "not_Beijing"
      }
    ]
  }
```


- **标准输出样本**: `[('Carol', 10001)]`
- **学生输出样本**: `[('Carol', 8)]`

递归 CTE 终止边界与沙盒熔断

- **策略说明**：静态检测 `WITH RECURSIVE` 结构。除了在自引用序列上产生离散数据校验终止边界外，还在 SQLite 沙盒执行时启用虚拟机周期计数器（Progress Handler），将指令周期锁定在 10 万个以内。一旦死循环立即熔断，防止系统被学生错误 SQL 挂死。
- **Schema**: `dummy(id)`
- **标准答案 SQL**:

代码块

```
WITH RECURSIVE nums AS (SELECT 1 AS n UNION ALL SELECT n + 1 AS n FROM nums WHERE n < 3) SELECT n FROM nums;
```

- **学生作答 SQL**:

代码块

```
WITH RECURSIVE nums AS (SELECT 1 AS n UNION ALL SELECT n + 1 AS n FROM nums WHERE n < 5) SELECT n FROM nums;
```

- **沙盒判定等价性**: `False`
- **归因 KP**: `['union', 'cte-recursive', 'where']`
- **策略检查结果**:
  a. `PASS` - standard/student=3/5, error=None
  b. `PASS` - expected KP=cte-recursive, actual=['union', 'cte-recursive', 'where']
- **动态生成的数据集**:

代码块


```
{
  "dummy": [
    {
      "id": 6
    },
    {
      "id": 7
    },
    {
      "id": 8
    },
    {
      "id": 1
    },
    {
      "id": 2
    },
```

```
      {
        "id": 3
      },
      {
        "id": 4
      },
      {
        "id": 5
      }
    ]
  }
```

- **标准输出样本**: `[(1,), (2,), (3,)]`
- **学生输出样本**: `[(1,), (2,), (3,), (4,), (5,)]`

3.

阶段一：SQL 算子覆盖与策略总结表

在 SQL 智能教学系统的阶段一（Observe）中，不同的 SQL 算子（对应关系代数算子或 DQL 子句）有着各自独立的 **AST 识别规则、动态造数（数据生成）策略** 与 **变分隔离测试机制**。

以下表格系统化整理了当前架构中运用和支持的所有核心算子及造数策略：

算子类别	算子/子句名称	SQL 语法表现	Sqlglot AST 节点	动态造数 (数据生成) 策略	变分隔离 (Mutation) 机制	映射知识点 ID (KP ID)
选择 (Selection)	`WHERE` (值过滤)	`WHERE col = 10` 等	`exp.Where`, `exp.Comparison`	**边界三态划分注入**：抽取谓词边界值 c ，在前三行分别注入 `[c, c + 1, c - 1]`，以确保完全覆盖均符合区 (T_{both})、差异区 (T_{diff}) 和均不符合区 ($T_{neither}$)，杜绝边界滑移的漏报。	替换变体：强行植入标答 `WHERE`；移除变体：移除学生 `WHERE`。	`where` (`COMP_VAL`)
空值过滤 (Null Filter)	`WHERE col IS NULL`	`WHERE col IS NULL`	`exp.Is`	专门检测 `exp.Is` 且包含 `exp.Nul`	伴随 `WHERE` 变体一同替换。	`comp-null` (`COMP_NULL`)

				`l` 的空值匹配项。确保在特定行注入 `None`，以验证学生是否混淆了 `col = NULL` 与 `col IS NULL`。		
投影 (Projection)	`SELECT`	`SELECT col1, col2`	`exp.Select`	动态识别引用的列名，仅针对目标列生成基础种子值（`_seed_value`），限制输出行数。	暂不进行单独的变体替换（通常受 `WHERE` / `JOIN` 错误连带影响）。	`select-basic`（`PROJ_COL`）
去重 (Duplicate)	`DISTINCT`	`SELECT DISTINCT col`	`exp.Distinct`	**去重探针**：在 Row 0 和 Row 1 的非主键列（排除 ID/SSN 核心主键，但包含 `course_id` / `dept_name` 等外键或普通列）复制生成完全重复的值，以此检验 `DISTINCT` 缺失。	暂不进行单独的子句变体替换。	`distinct`（`DISTINCT_SET`）
连接 (Join)	`JOIN ON` / `USING`	`JOIN t2 ON t1.id = t2.id`	`exp.Join`	1. **拓扑对齐**：使用 `_join_group_key` 建立共享值池，防止空连接。 2. **join-group 内唯一偏移漂移**：对同一关联值池中的不同 `table.column` 分配确定性且尽量不重复的 offset，	连接条件变体：提取标答 `JOIN ON` 的 ON 条件（如 `std_on`）移植替换到学生 Join 中。	`join-on` / `join-inner` / `join-left` / `join-right` / `join-full`

				防止多外键列（如 `s_ID` 与 `i_ID`）值在行内相同而产生同构屏蔽；不再依赖简单 ASCII 求和是否碰巧不同。 3. **外连接悬浮元组设计**：对关系/子表（如 `takes`）最后一行强制赋 `None`，为父表保留非匹配行，从而区分 LEFT JOIN 与 INNER JOIN。		
分组 (Grouping)	`GROUP BY`	`GROUP BY col`	`exp.Group`	每张表默认生成 4~8 行，确保分组字段存在多个不同组合的行，暴露未分组或分组字段写错的情况。	替换变体：将标答的 `GROUP BY` 子句移植注入到学生 SQL 中进行沙盒复测。	`group-by` (`GB_SIMPLE`)
分组过滤 (Having)	`HAVING`	`HAVING COUNT(*) > 1`	`exp.Having`	**聚合边界三态造数**：对 `SUM/AVG/MIN/MAX` 直接控制每个分组的聚合指标，使分组分别落在 `c + 1`、`c`、`c - 1`；对 `COUNT` 不改写数值列，而是重排分组键，使组大小分别落在 `c + 1`、`c`、`c - 1`。	替换变体：强行植入标答 `HAVING`；移除变体：移除学生 `HAVING`。	`having` (`HV_SIMPLE`)

排序 (Sorting)	`ORDER BY`	`ORDER BY col ASC`	`exp.Order`	在 `_seed_value` 中生成具有单调递增/递减特征的数据序列。	替换变体：强行植入标答 `ORDER BY`；激活 **有序模式精确比对**（禁用无序的 Counter 比对）。	`order-by` (`SORT_ASC`)
限制 (Limiting)	`LIMIT` / `OFFSET`	`LIMIT 5`	`exp.Limit` / `exp.Offset`	限制输出列表的行数，与 standard_rows 的长度进行核对。	替换变体：替换学生 SQL 中的 `LIMIT` 参数值。	`limit` (`LIMIT_OFF`)
简单子查询 (Subquery)	非相关子查询	`WHERE col IN (SELECT...)`	`exp.Subquery` 等	提取子查询中的关联列，在父子表之间建立主外键或数据范围 of 重合。	结合 `WHERE` 算子作为条件的一部分进行替换变体测试。	`subquery-scalar` / `subquery-in` / `subquery-exists`
相关子查询 (Corr-Subquery)	关联子查询	`WHERE s.ID IN (SELECT...)`	`exp.Subquery` 带有父表引用	静态检查嵌套子查询的列名，识别是否引用了外层主表的表名或别名。并在造数时确保内外层列具有交叉数据。	结合 `WHERE` 算子作为条件的一部分进行替换变体测试。	`subquery-correlated` (`SUB_COR`)
简单 CTE (CTE)	公用表表达式	`WITH temp AS (...)`	`exp.CTE` 非递归	提取 CTE 内外层 SQL 引用到的底层基表、连接键与谓词约束，并只生成这些基表数据；`WITH` 临时结果由 SQLite 沙盒原生执行，从而验证 CTE 基表约束能否传递到最终查询。	暂不进行单独变体替换。	`cte` (`CTE_SIMPLE`)

递归 CTE (Rec-CTE)	递归公用表表达式	`WITH RECURSIVE x AS (...)`	`exp.CTE` 递归或自引用	识别自引用递归并启动**安全沙盒熔断机制**（`conn.set_progress_handler` 限制 10 万周期），防止由于数据成环引起的无限递归挂死。	暂不进行单独变体替换。	`cte-recursive`（`CTE_RECURSIVE`）
并集运算 (Union)	并集操作	`SELECT... UNION SELECT...`	`exp.Union`	分别对 UNION 两侧的查询提取谓词约束，合成多表模拟数据并在沙盒中校验输出元组。	暂不进行单独的集合算子变体替换。	`union`（`SET_UNION`）
交集运算 (Intersect)	交集操作	`SELECT... INTERSECT SELECT...`	`exp.Intersect`	同并集操作，分别提取两侧的约束并在沙盒中校验。	暂不进行单独的集合算子变体替换。	`intersect`（`SET_INTERSECT`）
差集运算 (Except)	差集操作	`SELECT... EXCEPT SELECT...`	`exp.Except`	关系差集操作。抽取 EXCEPT 右侧的过滤条件，并在数据中生成排他数据行以校验结果。	暂不进行单独的集合算子变体替换。	`except`（`SET_EXCEPT`）
条件分支 (Conditional)	`CASE WHEN`	`CASE WHEN... THEN... END`	`exp.Case`	针对 CASE WHEN 条件块中的各分支条件，分别产生匹配条件的模拟数据以遍历各计算分支。	暂不进行变体替换。	`case`（`CASE_SEARCH`）
窗口函数 (Windowing)	`OVER`	`ROW_NUMBER() OVER (...)`	`exp.Window`	提取窗口排序列与分组列，在数据行中产生重复分组和乱序行，以检验排名的正确性。	暂不进行变体替换。	`window-row-number`（`WIN_OVER`）

1.上下文无关语法

2.算子生成数据出现问题标注

3.解析到算子

https://dbgroup.cs.tsinghua.edu.cn/jnwang/papers/vldb2025-parseval.pdf

sqlgolt

https://sqlglot.com/